Lead Finder Agent - Project Analysis

1. Project Overview

Lead Finder Agent is a full-stack lead generation and business intelligence platform that automatically discovers, analyzes, and enriches business leads from Google Maps and business directories.

Purpose: Automate the lead generation process by scraping business directories, analyzing websites for contact information, and providing AI-powered insights about business potential and outreach strategies.

2. Technology Stack

Frontend (frontend/)

Component	Technology
Framework	Next.js 16 (App Router)
Language	TypeScript 5
Styling	Tailwind CSS 4
Icons	Lucide React
State Management	Zustand (with localStorage/sessionStorage persistence)
Data Fetching	TanStack React Query v5
UI Components	Radix UI primitives, custom components
Forms	React Hook Form with Zod validation
Animations	Framer Motion
Notifications	Sonner

Backend (backend/)

Component	Technology
Framework	FastAPI
Language	Python 3.12+
Database	MongoDB (Motor - async driver)
Data Validation	Pydantic v2
Web Scraping	Playwright
Background Jobs	APScheduler
Security	PyJWT, Passlib (bcrypt), cryptography (Fernet)
Email/WhatsApp	Custom SMTP and WhatsApp API integration
AI/ML	OpenAI integration (custom provider architecture)
Reports	ReportLab, WeasyPrint, Jinja2, Pandas

3. Project Structure

Backend (backend/)

```
backend/
├── app/
│   ├── api/
│   │   ├── routes/          # 25+ API route modules
│   │   │   ├── auth.py, leads.py, campaigns.py
│   │   │   ├── ai.py, whatsapp.py, email.py
│   │   │   ├── crm.py, intelligence.py, reports.py
│   │   │   └── ... (25+ total)
│   │   ├── dependencies.py  # Auth, permissions, middleware
│   ├── core/
│   │   ├── config.py        # Settings & environment variables
│   │   ├── security.py      # JWT, password hashing
│   │   ├── exceptions.py    # Custom exception classes
│   │   ├── logger.py        # Structured logging
│   │   ├── middleware.py    # Security headers, rate limiting
│   │   └── state_machine/   # AI analysis workflow states
│   ├── db/
│   │   ├── database.py      # MongoDB connection + health monitor
│   │   └── init_db.py       # Admin user & index creation
│   ├── models/              # 45 Pydantic schemas
│   │   ├── lead.py          # Lead document model
│   │   ├── ai_analysis.py   # AI analysis workflow
│   │   ├── ai.py, campaign.py, audit.py
│   │   └── ... (45+ total)
│   ├── repositories/        # Data access layer
│   ├── services/            # 100+ business logic modules
│   │   ├── ai/              # AI analysis providers
│   │   ├── campaigns/       # Email & WhatsApp campaigns
│   │   ├── whatsapp/        # WhatsApp automation
│   │   ├── reports/         # Report generation
│   │   ├── search_engine/   # Google Maps scraper
│   │   └── ... (100+ total)
│   ├── main.py              # FastAPI app entrypoint
│   └── websockets.py        # Real-time WebSocket handling
├── tests/                   # Pytest test suite (30+ files)
├── scripts/                 # 20+ utility scripts
├── .venv/                   # Python virtual environment
└── requirements.txt         # Python dependencies
```

Frontend (frontend/)

```
frontend/
├── app/
│   ├── (dashboard)/          # Authenticated routes
│   │   ├── dashboard/        # Main dashboard page
│   │   ├── leads/            # Lead management
│   │   ├── search/           # Google Maps search interface
│   │   ├── campaigns/        # Email & WhatsApp campaigns
│   │   ├── crm/              # CRM pipeline board
│   │   ├── intelligence/     # Business intelligence
│   │   ├── settings/         # User settings
│   │   └── admin/            # Admin panel
│   ├── login/                # Authentication page
│   ├── layout.tsx            # Root layout
│   └── page.tsx              # Home page (redirects)
├── src/
│   ├── components/           # Reusable React components
│   ├── features/
│   │   └── auth/             # Auth state management
│   ├── lib/                  # Utilities (axios, queryClient)
│   └── types/                # TypeScript interfaces
├── public/                   # Static assets
└── package.json              # NPM dependencies
```

4. Key Features

Lead Discovery & Scraping

- Google Maps search with category & geo-targeting
- Regional directory scraping
- Advanced search filters (rating, reviews, category)
- Smart pagination and deduplication

Website Analysis

- Playwright-based web crawling
- Contact information extraction (emails, phones, WhatsApp)
- Social media profile detection
- Technology stack identification (CMS, frameworks)
- SEO metrics analysis

AI-Powered Intelligence

- **AI Analysis Pipeline:** Multi-stage analysis (collecting → analyzing → scoring → saving)
- **Contact Discovery:** Automated email/phone extraction from websites
- **Business Intelligence:** Lead scoring, opportunity assessment
- **Technology Detection:** CMS, hosting, analytics identification
- **Sales Intelligence:** Priority classification, talking points

Campaign Management

- Email campaigns with templates
- WhatsApp campaigns with templates
- Campaign scheduling and tracking
- Analytics & reporting

CRM Integration

- Pipeline management (Stages: New Lead → Qualified → Closed)
- Lead assignment to team members
- Activity tracking (calls, emails, meetings)
- Priority and status management

Data Export

- CSV/Excel export
- Report generation (PDF, DOCX, XLSX, HTML)
- Custom report templates
- Automated daily reports

5. Database Architecture

MongoDB Collections

Collection	Purpose
`leads`	Primary lead documents with AI analysis metadata
`users`	User accounts and authentication
`campaigns`	Email and WhatsApp campaign configurations
`templates`	Email/WhatsApp message templates
`search_sessions`	Saved search configurations
`ai_analysis_jobs`	AI analysis job queue
`ai_analysis_queues`	AI analysis queue management
`advanced_analysis_queue`	Advanced analysis job queue
`worker_nodes`	Background worker status tracking
`job_logs`	Unified job logging
`backup_data`	Exported data backups
`tags`	Lead tagging system
`crm_activities`	CRM activity log
`intelligence_daily`	Daily aggregate intelligence data

Lead Document Schema (Python)

```
LeadDocument = {
  # Basic Business Info
  "name": str,
  "phone": str,
  "email": str,
  "website": str,
  "address": str,
  "city/state/country": str,
  "category": str,
  "google_rating": float,

  # AI Analysis (nested)
  "ai_analysis": {
    "analysis_status": "WAITING|QUEUED|RUNNING|COMPLETED|FAILED",
    "overall_score": int,
    "seo_score": int,
    "performance_score": int,
    "recommendations": List[str],
    "emails": List[str],
    "phones": List[str],
    "technology_detected": List[str],
    # ... many more fields
  },

  # Enriched Data (nested)
  "contact_discovery": { ... },
  "technology_detection": { ... },
  "seo": { ... },
  "business": { ... },

  # CRM Fields
  "pipeline_stage": str,
  "priority": str,
  "owner_id": str,

  # Metadata
  "user_id": str,
  "created_at": datetime,
  "tags": List[str],
  "is_favorite": bool
}
```

6. Background Workers & Scheduler

Workers (started on backend startup)

Worker	Purpose
`CampaignQueueWorker`	Email & WhatsApp campaign processing
`email_worker_instance`	Email dispatch queue
`EnhancedWAQueueWorker`	WhatsApp message dispatch
`WADesktopQueueWorker`	WhatsApp AI desktop automation
`EmailAIQueueWorker`	AI-powered email analysis
`AIQueueWorker`	AI analysis queue processing
`AdvancedAnalysisWorker`	Complex deep-dive analysis
`AutoReportWorker`	Automated report generation
`CRMAutomationWorker`	CRM workflow automation
`EnrichmentQueueWorker`	Lead enrichment processing

APScheduler Jobs

Job	Frequency	Purpose
`emit_heartbeat`	Every 30s	Worker health monitoring
`daily_aggregate`	Daily (midnight)	Daily aggregate snapshots
`daily_cleanup`	Daily (1 AM)	Data cleanup & archival

7. Authentication & Security

Authentication Flow

- User logs in with email/password
- JWT access token (15 min expiry) + refresh token (7 day expiry) issued
- Tokens stored in localStorage/sessionStorage (based on "remember me" option)
- Automatic token refresh on 401 errors
- Role-based access control (Admin, User)

Security Features

- JWT authentication
- Password hashing (bcrypt)
- Fernet encryption for sensitive API keys
- CORS configuration (environment-controlled origins)
- Rate limiting (slowapi)
- Security headers middleware
- Request size limits (10MB max)

8. AI Analysis Pipeline

Workflow State Machine

```
PENDING → RUNNING → COLLECTING → ANALYZING → SCORING → SAVING → SUCCESS
                                                    ↘ FAILED (on error)
```

- **Collection Stage:** Crawl website, discover pages, extract contacts
- **Analysis Stage:** SEO analysis, performance metrics, design audit
- **Scoring Stage:** Calculate business intelligence scores
- **Saving Stage:** Persist results to MongoDB

AI Providers

- **OpenAI Provider:** Primary AI provider (GPT models)
- **Provider Factory:** Dynamic provider selection based on configuration

9. Frontend Architecture

State Management

- **Zustand Store:** `useAuthStore` with persist middleware
- **Storage Strategy:** localStorage (remember me) vs sessionStorage
- **React Query:** Server state management with auto-caching

Key Routes

Route	Purpose
`/dashboard`	Main dashboard with stats
`/dashboard/search`	Google Maps search interface
`/dashboard/leads`	Lead management grid
`/dashboard/campaigns`	Campaign management
`/dashboard/crm`	CRM pipeline board
`/dashboard/intelligence`	Business intelligence
`/dashboard/admin`	Admin panel

10. Deployment & Configuration

Environment Variables

Backend (.env)

```
MONGODB_URL=mongodb://localhost:27017/lead_finder_agent
SECRET_KEY=your-secure-secret-key
ENCRYPTION_KEY=your-fernet-key
ADMIN_EMAIL=your-admin-email
ADMIN_PASSWORD=your-admin-password
```

Frontend (.env)

```
NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api/v1
```

Deployment Options

- **Development:** `npm run dev` + `uvicorn app.main:app --reload`
- **Static Export:** Next.js static export for Netlify/Vercel
- **Production:** Build + serve with PM2 or similar

11. Testing & Quality Assurance

Test Suite (backend/tests/)

30+ test files covering: AI analysis logic, API endpoints, Authentication, Lead models, Search engine, Extraction pipeline, CRM functionality, Production validation.

Test Framework

- Pytest with asyncio support
- Async database fixtures
- Production validation scripts

12. Key Strengths

- **Scalable Architecture:** Background workers, task queues, health monitoring
- **Comprehensive AI Pipeline:** Multi-stage analysis with fallback mechanisms
- **Production-Ready:** Circuit breakers, connection pooling, graceful shutdown
- **Modular Design:** Clean separation of concerns between API, services, models
- **Real-time Updates:** WebSocket integration for live progress tracking
- **Reporting Suite:** Multiple export formats (PDF, Excel, CSV, DOCX)
- **CRM Integration:** Full pipeline management with activity tracking
- **Security-First:** JWT, encryption, rate limiting, input validation

13. Areas for Improvement

- **Documentation:** Could benefit from more comprehensive API documentation
- **Frontend Tests:** No visible frontend testing setup (Jest/Cypress)
- **CI/CD:** No visible CI/CD configuration (GitHub Actions, GitLab CI)
- **Monitoring:** Could add APM (Datadog, Sentry) for production monitoring
- **Caching:** Could add Redis for session caching and frequently accessed data

14. Running the Project

Back-End Commands

```
python3 -m venv .venv
source .venv/bin/activate  # (or .venv\Scripts\activate on Windows)
pip install -r requirements.txt
python3 -m pip install playwright
python3 -m playwright install chromium
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Front-End Commands

```
npm install
npm run build
npm run dev
```

15. Summary

Lead Finder Agent is a sophisticated lead generation platform with:

- **Dual-stack architecture:** Next.js frontend + FastAPI backend
- **MongoDB persistence:** Rich document model with nested enrichment
- **AI-powered insights:** Multi-stage analysis with OpenAI integration
- **Automation engine:** Background workers for campaigns and enrichment
- **Comprehensive feature set:** From scraping to CRM to reporting

This is a production-grade system with solid engineering practices, good code organization, and comprehensive business logic. The architecture supports scaling through background workers, task queues, and database optimization.

DigitalOcean Droplet Deployment Guide

1. Provision the Droplet

Setting	Value
Image	Ubuntu 22.04 LTS
Plan	$24/mo (4GB RAM, 4 vCPUs) or higher
Authentication	SSH key (recommended) or password
Hostname	lead-finder-server
Datacenter Region	Choose closest to your users

Create Droplet via CLI (optional)

```
doctl compute droplet create lead-finder-server \
  --image ubuntu-22-04-x64 \
  --size s-2vcpu-2gb \
  --region nyc3 \
  --enable-monitoring \
  --ssh-keys ~/.ssh/id_rsa.pub
```

2. Initial Server Setup

Connect to your droplet & Update

```
ssh root@YOUR_DROPLET_IP
apt update && apt upgrade -y
```

Create Non-Root User & Firewall

```
adduser yourusername
usermod -aG sudo yourusername
ufw allow OpenSSH
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
```

3. Install Backend Dependencies

Switch user and install packages

```
su - yourusername

# Install Python 3.12 and required tools
sudo apt install -y python3.12 python3.12-venv python3.12-dev
sudo apt install -y build-essential libssl-dev libffi-dev
sudo apt install -y libpq-dev libjpeg-dev zlib1g-dev
sudo apt install -y libcurl4-openssl-dev libssl-dev
```

Install MongoDB

```
sudo apt install -y wget gnupg curl
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | sudo gpg --dearmor -o
echo "deb [ signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ] https://repo.mongo

sudo apt update
sudo apt install -y mongodb-org
sudo systemctl enable mongod
sudo systemctl start mongod
```

Install Playwright Browsers

```
sudo apt install -y libglib2.0-0 libnss3 libatk1.0-0 libatk-bridge2.0-0 libcups2 \
  libdrm2 libdbus-1-3 libxkbcommon0 libxcomposite1 libxdamage1 libxfixes3 \
  libxrandr2 libgbm1 libasound2 libpango-1.0-0 libcairo2 libgdk-pixbuf2.0-0

pip install playwright
python -m playwright install chromium
```

Install Node.js & NGINX

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo apt install -y nginx
sudo systemctl enable nginx
sudo systemctl start nginx
sudo apt install -y certbot python3-certbot-nginx
```

4. Backend Deployment Steps

```
cd ~
git clone YOUR_GITHUB_REPO_URL lead-finder-project
cd lead-finder-project/backend
python3.12 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
cp .env.example .env
```

.env Configuration for Production

```
PROJECT_NAME="Lead Finder Agent API"
VERSION="1.0.0"
API_V1_STR="/api/v1"
MONGODB_URL=mongodb://localhost:27017/lead_finder_agent
DATABASE_NAME=lead_finder_agent

# Run `python -c "import secrets; print(secrets.token_urlsafe(32))"`
SECRET_KEY=your_secure_random_string

# Run `python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key()
ENCRYPTION_KEY=your_fernet_key
```

5. Create Systemd Service (Backend)

```
sudo nano /etc/systemd/system/lead-finder-backend.service
```

```
[Unit]
Description=Lead Finder Agent Backend API
After=network.target mongod.service

[Service]
Type=simple
User=yourusername
WorkingDirectory=/home/yourusername/lead-finder-project/backend
Environment="PATH=/home/yourusername/lead-finder-project/backend/.venv/bin"
ExecStart=/home/yourusername/lead-finder-project/backend/.venv/bin/uvicorn app.main:a
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

6. Frontend Deployment

```
cd ~/lead-finder-project/frontend
npm install
npm run build
```

```
sudo nano /etc/systemd/system/lead-finder-frontend.service
```

```
[Unit]
Description=Lead Finder Agent Frontend
After=network.target

[Service]
Type=simple
User=yourusername
WorkingDirectory=/home/yourusername/lead-finder-project/frontend
ExecStart=/usr/bin/node --max-old-space-size=4096 /home/yourusername/lead-finder-proj
Restart=always
RestartSec=10
Environment="NODE_ENV=production"
Environment="NEXT_PUBLIC_API_URL=https://api.yourdomain.com/api/v1"

[Install]
WantedBy=multi-user.target
```

7. Configure NGINX Reverse Proxy

```
sudo nano /etc/nginx/sites-available/lead-finder
```

```
upstream frontend { server 127.0.0.1:3000; }
upstream backend { server 127.0.0.1:8000; }

server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    location / {
        proxy_pass http://frontend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
    }

    location /api/v1/ {
        proxy_pass http://backend/api/v1/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_read_timeout 86400; # For WebSockets
    }
}
```

Enable and start NGINX:

```
sudo ln -s /etc/nginx/sites-available/lead-finder /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

8. SSL Certificate with Let's Encrypt

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

9. Security Hardening & Backups

```
sudo apt install -y fail2ban
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
```

Backup Script (cron example)

```
#!/bin/bash
# ~/backup.sh
mongodump --db lead_finder_agent --out /home/yourusername/backups/$(date +%Y%m%d)
tar -czf /home/yourusername/backups/frontend_$(date +%Y%m%d).tar.gz /home/yourusernam
find /home/yourusername/backups -type f -mtime +30 -delete
```

10. Performance Optimization (NGINX Tuning)

```
# /etc/nginx/nginx.conf inside http block
gzip on;
gzip_vary on;
gzip_min_length 1024;
gzip_proxied any;
gzip_types text/plain text/css text/xml text/javascript application/javascript applic
```

11. Monitoring and Maintenance

- **Logs Backend:** `sudo journalctl -u lead-finder-backend -f`
- **Logs Frontend:** `sudo journalctl -u lead-finder-frontend -f`
- **Status Checking:** `htop`, `df -h`, `free -h`
- **MongoDB Clear Cache:** `mongosh --eval "db.adminCommand('flushCache')"`